NCUSCC 考核报告

整理日期：2026-10-18。代码与实验材料：CarolDestiny/ncuscc-hpc。

一、技术报告

1. 开发环境与工具

在 Ubuntu / WSL 2 中使用 VS Code 开发，语言为 C++17；使用 G++ 15.2.0、CMake 和 Python 完成编译、实验与校验，Git / GitHub 管理代码。第二题另在 i5-8250U Ubuntu 物理机上分析性能。

2. 第一题：负载生成与指标采集

工具与实现： 使用 C++ 线程生成可配置的 CPU 负载，通过 Linux /proc 定时采集 CPU、内存、I/O 和上下文切换，以 JSONL 保存结果。CPU 使用率根据两次采样的时间差计算，使用 PID 和启动时间识别进程。

成果： Release 构建、真实负载与采集联动、JSONL 解析和目标 PID 检查通过。原有两项测试脚本缺失，未将其计为已通过。

3. 第二题：Stencil 与矩阵乘法分析

工具与实现： 使用 G++、Python 比较 Stencil 的按行/按列遍历及 GEMM 的普通/分块实现；在物理机上结合 perf stat、Cachegrind、向量化报告和 objdump 分析访问顺序、数据复用及 tile 大小。

成果： 历史物理机实验中，4096×4096、20 步 Stencil 的 row 相对 column 加速 43.57 倍；N=1024、tile=64 的 blocked GEMM 相对 naive 加速 26.72 倍。主实验采用预热后的 9 样本中位数。历史两机共 472 次逐元素校验通过；本次 WSL 复验的 14 组正常参数和 2 组错误参数检查通过。

4. 第三题：稳定日志筛选优化

工具与实现： 使用 CMake、CTest、Python、ASan / UBSan 和 objdump；保留原始基线，抽取筛选接口，每 64 条记录构造命中位图，全命中时连续复制，部分命中时按位置顺序写出，保留重复项和全部字段。

成果： 历史 WSL 测量的四组混合命中场景加速约 1.68–2.68 倍；全命中场景耗时增加 8.35%，说明优化存在额外位图成本。公开测试、优化版 3 组 CTest、1057 个 API 用例及 ASan / UBSan 检查通过。

5. 详细数据与复现材料

题目	GitHub 材料
第一题	采样、构建命令与完整校验输出
第二题	详细报告 • 汇总数据 • 原始样本及环境
第三题	详细报告 • 逐次测量 • 测试输出

数据与复现总索引还提供分析脚本、物理机硬件计数及统一构建命令。上述历史材料链接固定到归档提交，便于核对。

结果口径： 第二题性能数据来自 2026-10-12，第三题来自 2026-10-15；2026-10-18 进行了本地正确性复验。加速比针对计算核心，未执行正式平台验收。

AI 使用： 第一题主要自行编写；第二题代码分析由本人完成，实验及报告整理借助 AI；第三题实现、测试和报告使用 AI 辅助。

二、感想报告

从基础关卡说起

以前用 Git 的时候，基本就是把 commit 和 push 连着用。因为都是自己一个人做项目，很少接触分支管理。这次学会了查看、创建和对比分支，也学会了查看文件差异，对 Git 总算有了比“提交然后上传GitHub”更深入一点的了解。

Linux 基础命令这部分，对我来说比想象中困难。以前虽然用过 Ubuntu，但大多数时候都是通过 GUI 操作，这是第一次在只有终端的环境里处理文件。以前遇到需要执行的命令，也不太会自己写参数，通常都是找现成的资料直接复制。这次实际操作时还是不太熟练，不少命令和参数也借助了 AI，但至少开始接触、使用这些东西了。

这部分学到了用 mkdir 创建文件夹、用 touch 创建文件、用 cat 查看文件内容，也学会了用 nano 编辑文件。环境里好像没有找到 Vim，所以就先用 nano 了。整个过程感觉有点困难，加上容器里的终端还会一卡一卡的，操作起来不算顺手，不过最后还是把基础部分写完了。

进阶部分：C/C++ 方向

后面的进阶部分，我选择了 C/C++ 方向。之后有时间也想学学 CUDA，不过最近确实太忙了QAQ

第一题基本是自己手写的。第二题的代码分析是自己做的，但实际的实验报告就主要借助 AI 了，因为我对这一方面的分析和报告写法还不太熟悉。

进阶部分的代码都是在 WSL + VS Code 里完成的。之前容器申请不到再加上 VS Code 有图形化界面，更舒服一些。

之前学过一些 CMake 的构建命令，但没有真正动手操作过。平时在 Windows 的 IDE 里，生成和构建通常直接点一下就完成了，不太需要自己关心这些步骤。这次实际用到之后，才发现还是有点困难。

代码本身还好，但英文注释读起来真的有点困难。:(